Network modeling – AY 2026/2027

Homework 2 – Composite shaping of Zoom uplink traffic

Alessandro Vulcu – ID 2185190, University of Padua

Abstract

I implement a simulated-time token bucket followed by a constant-rate leaky bucket, using the empirical Zoom source fitted in Homework 1. A grid search balances rate reservations and storage under explicit loss and delay budgets. Independent 30-minute replications confirm the selected configuration. Results describe the fitted traffic model and shaping contribution to QoS, rather than total call quality.

1 Source and implementation

Homework 1 selected a camera-on outbound Zoom UDP flow. The first 180 s formed the training sample (19,645 packets); the next 120 s were held out. Sizes are captured Ethernet frame lengths, not media payload. I reuse the training CSV, with a 1 ms clustering threshold and both boundary clusters excluded. The resulting 7,233 complete cycles store $(D_i, \{U_{i,k}, L_{i,k}\}_{k=1}^{B_i})$: cluster interval, packet offsets, and lengths. Uniform independent sampling of whole tuples preserves dependence between interval, count, and lengths. The fitted mean rate is 865.95 kbit/s; the maximum packet is 1,341 B. No new capture or refitting on validation is needed.

To meet this assignment's simultaneous-batch convention, all B_i packets are emitted at the cycle epoch, on separate rows with identical timestamps. The next epoch is advanced by D_i. This retains the cluster-interval, batch-size, and length distributions, while deliberately removing the short within-cluster offsets used in Homework 1. A separate option restores those offsets for sensitivity analysis. The source stops on $[0, T)$ and assigns consecutive unique packet identifiers.

`source.py` generates the input file; `shaper.py` reads it and writes the output. Both use three whitespace-separated fields: identifier, timestamp in seconds, length in bytes. Input times are generation times; output times are transmission completion times. IDs and lengths survive unchanged. Dropped IDs appear only in a separate drop log. No payload or wall-clock waiting is involved.

The token bucket starts full, with real-valued continuous tokens:

$$q(t+\Delta) = \min\{b, q(t) + \rho\Delta\}, \qquad t_{\text{eligible}} = t + \max\{0, L_{\text{head}} - q(t)\}/\rho.$$

One token covers one byte. Only the FIFO head can leave; a transfer consumes its entire length. Packets exceeding b are explicitly dropped. Insufficient tokens cause FIFO buffering in B_i; insufficient free storage causes a tail drop. There is no backpressure: a transfer to a full LBF queue is discarded and its tokens remain consumed. The LBF transmits one packet at a time for L/μ seconds, non-preemptively; its waiting queue has capacity B_o. Queue capacities count waiting bytes; the packet in service is excluded. An idle transmitter can therefore accept a packet even with $B_o = 0$.

The event clock jumps to the next arrival, head eligibility, or completion. At equal times it handles LBF completion first, then eligible queued TBF packets, then input rows in file order. Buffered packets cannot be bypassed. Both queues drain after source shutdown, avoiding artificial terminal loss. Eight automated checks cover exact timing, FIFO, both overflows, oversize rejection, input validation, source reproduction, conservation, and the TBF release envelope. All pass.

2 QoS budgets and experimental method

Zoom video is interactive: excessive waiting harms responsiveness; variation in delay raises receiver buffering needs, and loss removes video data. I propose the shaping-only budgets in Table 1, leaving room for propagation, other queues, codec processing, and playout. These are experimental design targets, not vendor requirements or an end-to-end quality guarantee. Mean delay measures typical waiting, the 95th and 99th percentiles expose tails, and loss prevents misleadingly good delay results obtained by discarding traffic. Byte delivery of at least 99% also prevents selective losses of large packets from hiding behind packet counts.

For each delivered packet j, $d_j = t_j^{\text{out}} - t_j^{\text{in}}$. Delay variation is $v_j = |d_j - d_{j-1}|$ for consecutive delivered packets, including pairs across lost IDs. Its 95th percentile is reported as jitter; it is not an RTP jitter estimator. Throughput counts completed bytes within $[0, T)$ divided by T; delivery fractions and delay include the final drain. Delays describe survivors only and are always interpreted alongside losses. All rates use captured bytes, not codec goodput.

I evaluate 192 configurations on one common 300 s trace, seed 2026:

$$\begin{aligned} \rho &\in \{110, 140, 180, 220\} \times 10^3 \text{ B/s}, & b &\in \{1341, 4096, 8192, 16384\} \text{ B}, \\ B_i &\in \{4096, 16384\} \text{ B}, & \mu &\in \{120, 220, 300\} \times 10^3 \text{ B/s}, \\ B_o &\in \{4096, 16384\} \text{ B}. \end{aligned}$$

Common input isolates parameter effects. Of these, 28 satisfy every budget. The initial lower-rate grid found no feasible setting, so the final grid includes higher rates; this exploration uses only the search trace. Among feasible candidates I minimize the explicit normalized resource score

$$C = \frac{\rho}{10^5} + \frac{\mu}{10^5} + \frac{b + B_i + B_o}{16384}.$$

Each rate reservation and the total storage carry equal unit weights at the stated scales. Bucket capacity counts as a burst allowance, not literal stored payload. The score expresses a design preference, not a monetary cost. A positive weight on every resource gives a non-dominated feasible grid choice; it is not a global optimum or a unique preference-independent solution.

I then freeze the choice and run ten independent 1,800 s replications, seeds 3026–3035, without reselecting on those results. Each run contains about 196,000 packets, or approximately 72,000 sampled cycles, and lasts ten times the training window. I retain full-start transient behavior because a call begins with empty queues and a full token bucket. The long duration makes this short transient small. All ten runs satisfy every budget. Table 1 gives the mean of each run-level statistic and a Student-t 95% confidence interval, $\bar{x} \pm t_{0.975,9} s_x / \sqrt{10}$. For quantiles this estimates the mean run quantile, not a confidence interval for an individual packet or a pooled quantile. Replications, not correlated packets, are the independent statistical units.

Table 1: Independent confirmation: means and 95% confidence half-widths.

KPI	Proposed budget	Mean $\pm$ half-width
Packet loss (%)	≤ 1	0.169 ± 0.014
Delivered bytes (%)	≥ 99	99.805 ± 0.016
Mean delay (ms)	≤ 30	16.712 ± 0.071
95th delay percentile (ms)	≤ 50	44.889 ± 0.294
99th delay percentile (ms)	≤ 100	68.280 ± 0.650
95th delay-variation percentile (ms)	≤ 30	18.396 ± 0.055
Offered rate (kbit/s)	Descriptive	865.196 ± 1.774
Output throughput (kbit/s)	Descriptive	863.502 ± 1.787

3 Results, trade-offs, and limits

The selected parameters are

$$(\rho, b, B_i, \mu, B_o) = (180000, 4096, 16384, 220000, 4096), \qquad C = 5.50.$$

This reserves 1.44 Mbit/s at the TBF and a 1.76 Mbit/s serialization rate. Total waiting storage is 20,480 B; adding the burst allowance gives 24,576 resource units. Both rates exceed the roughly 0.866 Mbit/s mean source load: short bursts, not only mean-rate stability, govern the delay budget. The full bucket admits a small burst before enforcing the long-term rate; the transmitter then serializes it.

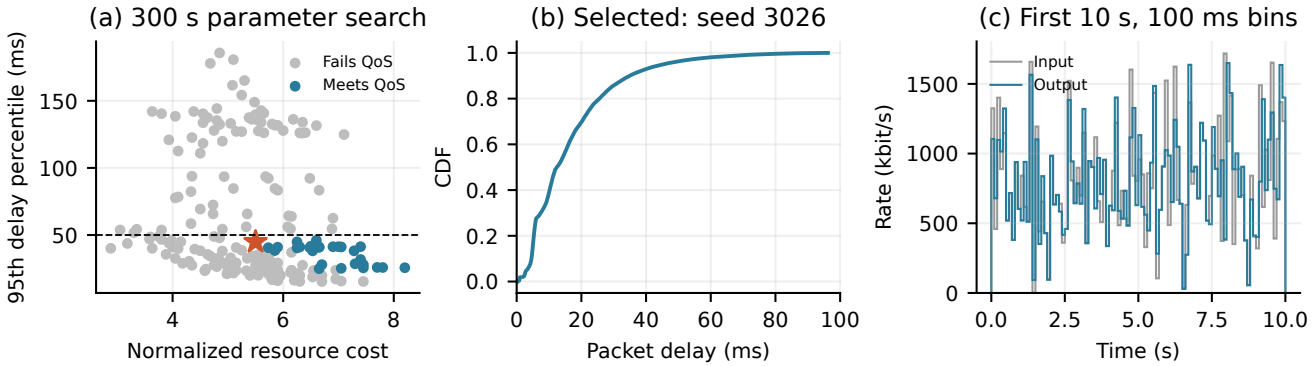


Figure 1: Search trade-off, delivered-packet delay distribution, and input/output rates for the selected setting. Star: minimum-score feasible candidate. Rate samples use 100 ms bins; packet completions can still cluster around bin boundaries.

The representative delivered trace (seed 3026) contains 196,181 of 196,556 packets, with 375 input-buffer drops and no output-buffer drops. Peak input occupancy is 16384 B and peak output waiting occupancy is 4038 B. The small output queue works because the TBF spreads the offered bursts before the faster transmitter. A bigger b can relieve input waiting but also pushes a larger burst into the output queue; increasing B_i prevents some losses while allowing longer delay. Increasing B_o cannot recover packets already discarded at the input. Higher ρ reduces token waiting; higher μ shortens serialization and output waiting. Their benefits depend on which stage is the bottleneck.

Table 2: Controls and robustness checks, using seed 3026 where synthetic.

Case	Loss (%)	p95 delay (ms)	Output (kbit/s)
Underprovisioned	29.16	48.98	587.15
Large buffers, low rates	1.10	546.62	855.08
Original within-batch offsets	0.19	45.28	864.03
Held-out 120 s capture	0.04	31.23	856.88

The underprovisioned control uses $(90000, 1341, 4096, 100000, 4096)$: low delay alone hides severe losses. A low-rate setting with large buffers, $(110000, 8192, 65536, 120000, 65536)$, still fails QoS and raises delay substantially. More storage alone is therefore insufficient. Restoring Homework 1's packet offsets barely changes the representative result. Processing the independent held-out capture also meets all budgets, but this short check does not replace the longer model runs.

The confidence intervals quantify Monte Carlo variability conditional on the fitted empirical source. They exclude uncertainty from the 180 s training sample, different users or network paths, and the missing slow temporal correlation identified in Homework 1. Independent cycle resampling loses that correlation and overstates some 100 ms variability; repeated synthetic hours do not create new observed evidence. The shaper also does not feed congestion signals back to Zoom's adaptive encoder. The resource choice is thus defensible for this model, grid, and cost preference; deployment would require more captures and end-to-end quality measurements.

Reproduction. `README.md` documents both module CLIs, training-data provenance, trace aliases, test commands, and full reproduction with `python experiments.py` followed by `python build_report.py`. Source code, measured tables, buffer/drop traces, figure source, and this report are included in the homework directory.